

PlayChat - Technical Specification for AI Agents

Document purpose: This file defines the system as a machine-readable implementation contract. The goal is to minimize interpretation during coding.

Scope: Web-based multiplayer game platform with rooms, chat, real-time game play, authentication, notifications, and per-user statistics.

1. Canonical terms

Use the following names exactly as written.

Term	Meaning
User	Authenticated account holder
Room	A shared space for chat and games
Room Owner	User who created the room
Room Member	User who joined the room as an active participant
Spectator	User who can observe the room but cannot make game moves
Game	A single playable match instance inside a room
Game Log	System message describing game events
Notification	In-app invitation or status message
General Stats	Aggregated totals across all games
Game Stats	Statistics for a single game type

2. System invariants

These rules are always true unless explicitly stated otherwise.

- **INV-001** A user can be in only one room at a time.
- **INV-002** A room can have only one active game at a time.
- **INV-003** Only the room owner can start a game.
- **INV-004** A user who joins after a game has started becomes a spectator.
- **INV-005** Spectators can view the room, chat, and game state, but cannot play moves.
- **INV-006** Old messages are not replayed to late joiners; only new messages from the join moment onward are visible.
- **INV-007** Game log messages must be visually separated from normal chat messages.
- **INV-008** If a player disconnects during an active game, the system waits 30 seconds before declaring defeat.
- **INV-009** Notifications are in-app only; no web push notification is used.
- **INV-010** There is no global leaderboard page.

3. Technology stack

Layer	Technology
Frontend	React 19 + TypeScript + Vite + Tailwind CSS
Realtime	Socket.IO v4
Backend	Node.js 22 + Fastify v5 + TypeScript
Database	PostgreSQL + Prisma
Cache	Redis
Auth	JWT + bcrypt
Frontend deploy	Vercel
Backend deploy	Railway

4. Game catalog

The platform includes exactly six games.

Game	Min players	Max players	Type
Tic-Tac-Toe	2	2	Turn-based 1v1
Memory Cards	2	4	Turn-based multiplayer
Connect Four	2	2	Turn-based 1v1
Wordle	2	4	Turn-based multiplayer
Rock Paper Scissors	2	2	Simultaneous 1v1
Number Guess	2	6	Turn-based multiplayer

4.1 Game engine rule

Every game must extend a common **GameEngine** abstract class.

Required responsibilities of **GameEngine**: - initialize state - validate moves - apply moves - detect win / draw / finish conditions - produce game log entries - expose current turn or active player information - handle disconnect timeout outcome

4.2 Game rules

- Only the room owner can start a game.
- A game can be started only when the room is in a valid waiting state.
- New users joining during an active game are spectators.
- A disconnected player has 30 seconds to reconnect.
- If the player does not reconnect in time, the opponent wins by timeout.

5. Room system

5.1 Room types

Type	Access rule
Public	Anyone can join
Private	Only users with the password can join

5.2 Room permissions

Room Owner can: - start the game - kick a user - close the room

Room Member can: - send chat messages - participate in the game if assigned as a player - view room updates

Spectator can: - view room updates - read incoming chat from join time onward
- watch the current game state

5.3 Room behavior rules

- A room join request must fail if the room is closed or inactive.
- A private room join request must verify the password before membership is created.
- When a user leaves a room, room state must update immediately.
- When the owner closes a room, all members must be removed or disconnected from it.
- Game log messages must be stored separately from normal chat messages using message type `GAME_LOG`.

6. Authentication

6.1 Supported auth methods

- Email + password registration
- Email + password login
- Google OAuth login
- Email verification after sign-up
- Password reset flow
- JWT session management

6.2 Auth rules

- Passwords must be stored only as hashes.
- JWT must be used for API and socket authentication.
- Email must be verified before full account use if verification is enabled.
- OAuth accounts may not require a local password hash.
- Password reset must use a time-limited token.

7. Notifications

7.1 Notification scope

- Notifications are shown only inside the application.
- The only required notification type is room invitation.

7.2 Notification rules

- A notification must record sender, receiver, room, and read state.
- Notification delivery must work without browser push.
- A notification is considered unread until marked read by the receiver.

8. Profile and statistics

8.1 Profile data

Every user profile should show: - display name - avatar - account status - total statistics - per-game statistics

8.2 Statistics model

`UserStats` must support two levels: - **GENERAL** for aggregate totals - one row per game type for game-specific results

Recommended fields: - `gamesPlayed` - `wins` - `losses` - `draws` - `roomsJoined`

8.3 Reporting rules

- No global leaderboard page.
- Stats must be available on the profile page.
- Game results must update statistics after game completion.

9. UI pages

9.1 Required pages

Page	Purpose
Login	Email/password and Google OAuth sign-in
Register	Sign-up form and password strength indicator
Dashboard	User rooms list
Explore	Public rooms discovery
Room	Chat and game panel side by side
Create Room	Public/private room creation form
Game Result	End-of-game score and summary
Profile	User profile and account settings

9.2 Design rules

- Theme: dark background #0a0a0a
- Typography: white text
- Style: minimal, clean, Linear/Vercel-like
- No purple/blue gradient as a primary visual theme
- Chat and game UI must remain visually separated inside Room page

10. Data model

10.1 Enum definitions

```
enum RoomType {  
    PUBLIC  
    PRIVATE  
}
```

```
enum MemberRole {  
    OWNER  
    MEMBER  
    SPECTATOR  
}
```

```
enum GameType {  
    GENERAL  
    TIC_TAC_TOE  
    MEMORY_CARDS  
    CONNECT_FOUR  
    WORDLE  
    ROCK_PAPER_SCISSORS  
    NUMBER_GUESS  
}
```

```
enum GameStatus {  
    WAITING  
    IN_PROGRESS  
    FINISHED  
}
```

```
enum PlayerRole {  
    PLAYER  
    SPECTATOR  
}
```

```
enum MessageType {  
    CHAT  
    GAME_LOG
```

}

10.2 User

Field	Type	Required	Notes
id	String	Yes	Primary key
email	String	Yes	Unique
passwordHash	String	No	Null for OAuth-only accounts
googleId	String	No	Unique
username	String	Yes	Unique
displayName	String	Yes	Public name
avatarUrl	String	No	Profile image URL
isVerified	Boolean	Yes	Email verification flag
currentRoomId	String	No	Current room pointer
createdAt	DateTime	Yes	Auto-generated
updatedAt	DateTime	Yes	Auto-updated

10.3 Room

Field	Type	Required	Notes
id	String	Yes	Primary key
name	String	Yes	Room title
description	String	No	Optional short text
type	RoomType	Yes	Public or private
passwordHash	String	No	Required only for private room
maxMembers	Int	Yes	Capacity limit
isActive	Boolean	Yes	Closed rooms are inactive
creatorId	String	Yes	Owner reference
currentGameId	String	No	Must point to the active game only
createdAt	DateTime	Yes	Auto-generated
updatedAt	DateTime	Yes	Auto-updated

10.4 RoomMember

Field	Type	Required	Notes
id	String	Yes	Primary key
roomId	String	Yes	Room reference
userId	String	Yes	User reference
role	MemberRole	Yes	OWNER, MEMBER, or SPECTATOR
joinedAt	DateTime	Yes	Join timestamp

Constraints: - unique(roomId, userId) - user can belong to only one active room at a time

10.5 Message

Field	Type	Required	Notes
id	String	Yes	Primary key
roomId	String	Yes	Room reference
userId	String	No	Null allowed for system messages
content	String	Yes	Message body
type	MessageType	Yes	CHAT or GAME_LOG
createdAt	DateTime	Yes	Auto-generated

Rules: - `userId = null` is allowed only for system/game log messages. - Game log messages must not be mixed with regular chat styling.

10.6 Game

Field	Type	Required	Notes
id	String	Yes	Primary key
roomId	String	Yes	Room reference
type	GameType	Yes	Game identifier
status	GameStatus	Yes	Waiting, in progress, finished
state	Json	Yes	Game-specific state
startedAt	DateTime	No	Set when game begins
createdAt	DateTime	Yes	Auto-generated

Recommended additions: - `winnerId` - `finishedAt` - `lastMoveAt`

10.7 GamePlayer

Field	Type	Required	Notes
id	String	Yes	Primary key
gameId	String	Yes	Game reference
userId	String	Yes	User reference
role	PlayerRole	Yes	PLAYER or SPECTATOR
symbol	String	No	Board symbol or game-specific label

Constraint: - `unique(gameId, userId)`

10.8 UserStats

Field	Type	Required	Notes
id	String	Yes	Primary key
userId	String	Yes	User reference
gameType	GameType	Yes	GENERAL means aggregate totals
gamesPlayed	Int	Yes	Total played count
wins	Int	Yes	Win count
losses	Int	No	Optional if supported
draws	Int	No	Optional if supported
roomsJoined	Int	No	Optional if supported

Constraint: - unique(userId, gameType)

10.9 Notification

Field	Type	Required	Notes
id	String	Yes	Primary key
userId	String	Yes	Receiver
senderId	String	Yes	Sender
roomId	String	Yes	Related room
isRead	Boolean	Yes	Read state
createdAt	DateTime	Yes	Auto-generated

11. API contract

11.1 REST endpoints

Auth (/api/auth)

Method	Path	Purpose
POST	/register	Create account
POST	/login	Login and return JWT
GET	/google	Start Google OAuth
GET	/google/callback	OAuth callback
POST	/verify-email	Verify email
POST	/forgot-password	Send reset email
POST	/reset-password	Set new password

Rooms (/api/rooms)

Method	Path	Purpose
GET	/	List public rooms
POST	/	Create room
GET	/:id	Get room details
POST	/:id/verify	Verify private room password
GET	/my-rooms	List user room history

Users (/api/users)

Method	Path	Purpose
GET	/:id	Get user profile
PUT	/me	Update profile
GET	/:id/stats	Get detailed statistics

Notifications (/api/notifications)

Method	Path	Purpose
GET	/	List invitation notifications
PUT	/:id/read	Mark one notification as read
PUT	/read-all	Mark all as read

11.2 REST rules

- REST is used for CRUD, auth, profile, and static data access.
- Real-time room chat and game actions are not handled by REST.
- Sensitive actions must be authenticated.

12. Socket contract

12.1 Authentication flow

Event	Direction	Purpose
connection	lifecycle	Socket connection established
authenticate	Client -> Server	Send JWT and authenticate socket
authenticated	Server -> Client	Authentication success ack
disconnect	lifecycle	Socket disconnected

12.2 Room events

Event	Direction	Purpose
room:join	Client -> Server	Join room request
room:joined	Server -> Client	Join success payload
room:user_joined	Server -> Client	Broadcast new member
room:leave	Client -> Server	Leave room
room:user_left	Server -> Client	Broadcast member left
room:kick	Client -> Server	Owner kicks target user
room:close	Client -> Server	Owner closes room
room:updated	Server -> Client	Broadcast room update

12.3 Chat events

Event	Direction	Purpose
message:send	Client -> Server	Send chat message
message:received	Server -> Client	Broadcast received message

12.4 Game events

Event	Direction	Purpose
game:start	Client -> Server	Start a game
game:move	Client -> Server	Send a move
game:state	Server -> Client	Broadcast current game state
game:end	Server -> Client	Broadcast game result

12.5 Notification events

Event	Direction	Purpose
notification:invite	Client -> Server / Server -> Client	Room invitation flow

12.6 Socket rules

- Socket authentication must use the same JWT identity as REST.
- All room/game events must validate room membership.
- Game state updates must be broadcast to room participants only.
- Spectators may receive state updates but cannot submit moves.
- Disconnect timeout must begin immediately after an active player disconnects.

13. State transitions

13.1 Game lifecycle

WAITING -> IN_PROGRESS -> FINISHED

Allowed transitions: - WAITING to IN_PROGRESS when the owner starts the game
- IN_PROGRESS to FINISHED when win, draw, surrender, or timeout occurs

Invalid transitions: - starting a game that is already in progress - submitting a move after game end - joining as player after game start

13.2 Room lifecycle

- create room
- join room
- leave room
- kick member
- close room
- reopen is not part of scope unless explicitly added later

14. Non-goals

The following are intentionally out of scope.

- global leaderboard page
- browser push notifications
- voice chat
- file sharing
- room history replay for late joiners
- multiple simultaneous room membership

15. Implementation notes for agents

- Use exact enum names and event names.
- Do not invent new game types.
- Do not rename **GENERAL** in **GameType**.
- Keep game logs separate from chat messages.
- Prefer explicit relations and constraints over implicit assumptions.
- When a rule says **must**, it is mandatory.
- When a rule says **recommended**, it is optional but preferred.